

```
In [1]: import numpy as np # import pakietu do obsługi wektorów, macierzy i innyc  
# dalej używane jest odniesienie np. wskazujące na użycie obiektów i meto  
import math
```

```
In [2]: # Wartość liczby pi  
np.pi
```

```
Out[2]: 3.141592653589793
```

```
In [3]: # obliczanie pierwiastka z podanej liczby  
np.sqrt(4)
```

```
Out[3]: 2.0
```

```
In [4]: # Dokumentacja dla arctan2  
help(np.arctan2)
```

Help on ufunc:

```
arctan2 = <ufunc 'arctan2'>
  arctan2(x1, x2, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature, extobj])
```

Element-wise arc tangent of ``x1/x2`` choosing the quadrant correctly.

The quadrant (i.e., branch) is chosen so that ``arctan2(x1, x2)`` is the signed angle in radians between the ray ending at the origin and passing through the point (1,0), and the ray ending at the origin and passing through the point (`x2`, `x1`). (Note the role reversal: the "y"-coordinate is the first function parameter, the "x"-coordinate is the second.) By IEEE convention, this function is defined for `x2` = +/-0 and for either or both of `x1` and `x2` = +/-inf (see Notes for specific values).

This function is not defined for complex-valued arguments; for the so-called argument of complex values, use `angle`.

Parameters

-----

x1 : array\_like, real-valued  
`y`-coordinates.

x2 : array\_like, real-valued  
`x`-coordinates.

If ``x1.shape != x2.shape``, they must be broadcastable to a commo

n

shape (which becomes the shape of the output).

out : ndarray, None, or tuple of ndarray and None, optional

A location into which the result is stored. If provided, it must h

ave

a shape that the inputs broadcast to. If not provided or None, a freshly-allocated array is returned. A tuple (possible only as a keyword argument) must have length equal to the number of outputs.

where : array\_like, optional

This condition is broadcast over the input. At locations where the condition is True, the `out` array will be set to the ufunc resul

t.

Elsewhere, the `out` array will retain its original value.

Note that if an uninitialized `out` array is created via the defau

lt

``out=None``, locations within it where the condition is False wil

l

remain uninitialized.

\*\*kwargs

For other keyword-only arguments, see the :ref:`ufunc docs <ufuncs.kwargs>`.

Returns

-----

angle : ndarray

Array of angles in radians, in the range ``[-pi, pi]``.

This is a scalar if both `x1` and `x2` are scalars.

See Also

-----

arctan, tan, angle

Notes

-----

\*arctan2\* is identical to the ``atan2`` function of the underlying C library. The following special values are defined in the C standard: [1]

```
=====
`x1`   `x2`   `arctan2(x1,x2)`
=====
+/- 0   +0     +/- 0
+/- 0   -0     +/- pi
  > 0   +/-inf +0 / +pi
  < 0   +/-inf -0 / -pi
+/-inf +inf    +/- (pi/4)
+/-inf -inf    +/- (3*pi/4)
=====
```

Note that +0 and -0 are distinct floating point numbers, as are +inf and -inf.

#### References

-----

.. [1] ISO/IEC standard 9899:1999, "Programming language C."

#### Examples

-----

Consider four points in different quadrants:

```
>>> x = np.array([-1, +1, +1, -1])
>>> y = np.array([-1, -1, +1, +1])
>>> np.arctan2(y, x) * 180 / np.pi
array([-135., -45., 45., 135.])
```

Note the order of the parameters. ``arctan2`` is defined also when ``x2``  
= 0

and at several other special points, obtaining values in the range ``[-pi, pi]``:

```
>>> np.arctan2([1., -1.], [0., 0.])
array([ 1.57079633, -1.57079633])
>>> np.arctan2([0., 0., np.inf], [+0., -0., np.inf])
array([0., 3.14159265, 0.78539816])
```

In [5]: `np.arctan2(4,4)`

Out[5]: 0.7853981633974483

In [6]: `np.arctan2(0,4)`

Out[6]: 0.0

In [7]: `np.arctan2(4,0)`

Out[7]: 1.5707963267948966

In [8]: `np.arctan2(-4,0)`

Out[8]: -1.5707963267948966

```
In [9]: np.arctan2(0, -4)
```

```
Out[9]: 3.141592653589793
```

```
In [10]: # Funkcja sinus. Argumentem wejściowym jest kąt w radianach.  
alpha = math.radians(30) # konwersja stopni na radiany  
np.sin(alpha)
```

```
Out[10]: 0.49999999999999994
```

```
In [11]: # Funkcja cosinus. Argumentem wejściowym jest kąt w radianach.  
alpha = math.radians(60) # konwersja stopni na radiany  
np.cos(alpha)
```

```
Out[11]: 0.50000000000000001
```

```
In [12]: # Utworzenie wektora składającego się z 3 elementów  
a = np.array([1,2,3])  
print(a)
```

```
[1 2 3]
```

```
In [13]: # Utworzenie macierzy o rozmiarze 3x2  
A = np.array([[1, 2],  
              [3, 4],  
              [5, 6]])  
  
print(A)  
print(f"Rozmiar macierzy: {A.shape}")
```

```
[[1 2]  
 [3 4]  
 [5 6]]
```

```
Rozmiar macierzy: (3, 2)
```

```
In [14]: # Utworzenie macierzy diagonalnej  
Adiag = np.diag([1,2,3,4])  
print(Adiag)
```

```
[[1 0 0 0]  
 [0 2 0 0]  
 [0 0 3 0]  
 [0 0 0 4]]
```

```
In [15]: # Transpozycja macierzy A  
np.transpose(A)
```

```
Out[15]: array([[1, 3, 5],  
               [2, 4, 6]])
```

```
In [16]: # Mnożenie macierzy  
M1 = np.array([[1,0,0],  
               [0,2,0],  
               [0,0,3]])  
M2 = np.array([[1,2,3],  
               [3,4,5],  
               [1,1,2]])  
  
W = M1 @ M2 # operator mnożenia macierzy  
print(W)
```

```
[[ 1  2  3]
 [ 6  8 10]
 [ 3  3  6]]
```

```
In [17]: # Wyznaczenie macierzy pseudoinwersji Moore'a-Penrose'a macierzy B
B = np.array([[1,0],
              [0,2],
              [0,5]])
Bpinv = np.linalg.pinv(B)
print(Bpinv)
```

```
[[1.         0.         0.         ]
 [0.         0.06896552 0.17241379]]
```

```
In [18]: # Wyznaczanie macierzy odwrotnej do macierzy B
B = np.array([[1,0,0],
              [0,2,0],
              [0,0,3]])

Binv = np.linalg.inv(B)
print(Binv)
print("Sprawdzenie:")
print(B @ Binv)
```

```
[[1.         0.         0.         ]
 [0.         0.5         0.         ]
 [0.         0.         0.33333333]]
```

Sprawdzenie:

```
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

```
In [19]: # konwersja danych z formatu numpy na listę.
B = np.array([[1,0,0],
              [0,2,0],
              [0,0,3]])
B_lista = B.tolist()
print(f"Typ danych B: {type(B)}")
print(B)
print("\n")
print(f"Typ danych B_lista: {type(B_lista)}")
print(B_lista)
```

Typ danych B: <class 'numpy.ndarray'>

```
[[1 0 0]
 [0 2 0]
 [0 0 3]]
```

Typ danych B\_lista: <class 'list'>

```
[[1, 0, 0], [0, 2, 0], [0, 0, 3]]
```

In [ ]:

In [ ]: